Tejiendo redes: arquitectura de software entre hilos y nodos

NexusAI 2.0: de la arquitectura a la red

Juan Carlos Benítez Rojas

Juan José Córdoba Eraso

Corporación Escuela Tecnológica del Oriente

Ingeniería de Software

Arquitectura de Software — Unidad 2

Leidy Johana Polo

11 de noviembre de 2026

Índice

Introducción 5

Contexto del proyecto 5

Propósito del diseño arquitectónico 5

Objetivos 6

Descripción del sistema distribuido 7

Tipo de sistema y dominio 7

Componentes funcionales 7

Supuestos y restricciones 8

Herramientas seleccionadas 9

Análisis y planificación del proyecto 10

Requisitos funcionales 10

Requisitos no funcionales 11

Relevancia de la comunicación entre nodos e hilos 11

Metas del sistema y métricas de éxito 13

Estrategia de trabajo, roles y cronograma 13

Modelado UML 15

Diagrama de componentes 16

Diagrama de secuencia 18

Diagrama de despliegue 20

Selección y justificación de patrones de diseño 21

Saga orquestada 21

Broker de mensajes 22

Mediator 22

Proxy y API Gateway 23

Circuit Breaker 23

Bulkhead 23

Observer distribuido 24

Strategy 24

Retry 24

Síntesis de patrones 25

Implementación y simulación 25

Estructura del código 25

Modelo de procesos, hilos y sincronización 26

Contrato HTTP y flujo de una transacción 27

Simulación de las etapas cognitivas 28

Puesta en marcha y reproducibilidad 29

Observabilidad y trazabilidad 29

Evaluación y ajustes 30

Metodología experimental 30

Escalabilidad horizontal 31

Robustez ante fallos 34

Costo por paso y sobrecarga de coordinación 36

Ajustes derivados de la medición 37

Cumplimiento de las metas 39

Limitaciones y validez 40

Conclusiones 40

Referencias 43

Anexo A. Contratos HTTP del prototipo 45

Anexo B. Extracto de la bitácora de eventos 47

Anexo C. Salida de la suite de evaluación 48

Anexo D. Reproducción del entorno 49

Anexo E. Enlaces de entrega 50

Introducción

Contexto del proyecto

La ingeniería de sistemas distribuidos y la computación orientada a servicios demandan una verificación rigurosa de las propiedades no funcionales. En entornos de nube, la red no es homogénea ni instantánea: las fallas de transporte, la latencia variable y la competencia por hilos de procesamiento representan condiciones cotidianas. Bajo este panorama, NexusAI 2.0 surge como la materialización ejecutable y experimental de una plataforma diseñada para la orquestación distribuida de pipelines cognitivos empresariales.

El proyecto retoma la arquitectura de NexusAI planteada en la fase conceptual previa y traslada sus premisas a un entorno de red operativo. Mientras que el diseño inicial definió los componentes lógicos fundamentales —Saga, Circuit Breaker, Bulkhead y API Gateway—, persistía la necesidad de validar su comportamiento dinámico bajo cargas concurrentes. NexusAI 2.0 supera esa frontera: transforma las especificaciones teóricas en procesos autónomos del sistema operativo, gobernados por contratos HTTP explícitos, sincronización multi-hilo y un bus asíncrono de eventos con auditoría persistente.

Propósito del diseño arquitectónico

El propósito central de la arquitectura es demostrar cómo una topología basada en servicios desacoplados puede garantizar simultáneamente tres atributos de calidad esenciales: escalabilidad horizontal, robustez ante fallos y mantenibilidad evolutiva (Bass et al., 2022). La escalabilidad se valida mediante la adición de nodos de cómputo sin estado compartido, permitiendo un incremento proporcional en el caudal transaccional. La robustez se alcanza confinando perturbaciones mediante aislamiento por mamparos (Bulkhead), disyuntores de circuito (Circuit Breaker) y recuperación de consistencia eventual mediante Sagas orquestadas

con compensación inversa. La mantenibilidad se preserva aislando la lógica interna tras contratos uniformes y observabilidad integral.

Estos atributos se evalúan cuantitativamente conforme al estándar ISO/IEC 25010 (International Organization for Standardization, 2011). Cada decisión de diseño —desde la elección de algoritmos de balanceo hasta la gestión de colas thread-safe— está respaldada por mediciones empíricas controladas que cuantifican su costo y beneficio en el sistema.

Objetivos

Objetivo general

Diseñar, modelar, implementar y evaluar un prototipo distribuido de alta eficiencia y resiliencia para la plataforma NexusAI 2.0, aplicando patrones arquitectónicos de concurrencia y comunicación inter-nodos que optimicen la escalabilidad horizontal y aseguren la continuidad del servicio ante degradaciones operativas.

Objetivos específicos

- Modelar formalmente la arquitectura mediante especificaciones UML rigurosas: un diagrama de componentes para interfaces y desacoplamiento, un diagrama de secuencia para la sincronización temporal entre hilos y nodos, y un diagrama de despliegue para la distribución física en red.
- Seleccionar y fundamentar técnicamente patrones de diseño orientados a sistemas concurrentes y distribuidos, contrastando sus ventajas frente a los costos operativos identificados en la literatura académica.
- Implementar un clúster funcional en Python estructurado en procesos autónomos, servidores HTTP multi-hilo, pools de despacho acotados y un broker de mensajería asíncrono con bitácora inmutable.

- Evaluar el desempeño del sistema mediante una batería de pruebas de carga y escenarios de caos controlado (inyección de latencia, nodos caídos, fallas de negocio y recuperación dinámica).
- Documentar las métricas de rendimiento observadas, identificar los cuellos de botella en la coordinación y aplicar refactorizaciones arquitectónicas medibles en el prototipo.

Descripción del sistema distribuido

Tipo de sistema y dominio

NexusAI 2.0 es un sistema distribuido orientado a servicios diseñado para procesar pipelines cognitivos complejos. Su función consiste en recibir transacciones omnicanal (consultas desde interfaces web, aplicaciones móviles o asistentes de mensajería) y ejecutarlas secuencialmente mediante tres etapas diferenciadas: clasificación inicial de intención y criticidad (*triage*), recuperación de contexto semántico aumentado (*rag*), y validación de políticas y salvaguardas éticas (*compliance*).

Adoptar una arquitectura de servicios desacoplados en lugar de un monolito responde a la naturaleza heterogénea de estas etapas. Cada fase exhibe perfiles computacionales distintos: el triage demanda baja latencia, la recuperación semántica consume intensivamente operaciones de entrada/salida y red, mientras que el cumplimiento normativo requiere validaciones deterministas estrictas. La separación en nodos independientes permite escalar el cómputo de forma modular, aislar eventuales fallos de infraestructura y auditar granularmente cada transición de estado sin bloquear el flujo global.

Componentes funcionales

El sistema organiza sus responsabilidades funcionales en cuatro subsistemas claramente delimitados:

1. **Subsistema de presentación y carga:** Comprende el generador de carga sintética concurrente y el tablero de observabilidad web. El generador somete al clúster a perfiles de demanda controlados; el tablero consume métricas agregadas en tiempo real para visualizar latencias, estado de disyuntores y rendimiento global.

2. **Subsistema de coordinación (Nodo 1):** Aloja el API Gateway, el orquestador de la Saga, el balanceador de carga y los Circuit Breakers. Este nodo centraliza el gobierno de la transacción: expone la interfaz REST unificada, preserva la máquina de estados transaccional y administra un pool de despacho dedicado para admitir clientes concurrentes.

3. **Subsistema de mensajería (Nodo 2):** Implementado en el broker de eventos. Administra tópicos en memoria (`events` y `audit`), gestiona colas acotadas con contrapresión (*backpressure*) y persiste cada evento en una bitácora JSONL inmutable, garantizando trazabilidad sin generar acoplamiento directo entre productores y consumidores.

4. **Subsistema de cómputo (Nodos 3, 4 y 5):** Conformado por nodos trabajadores (*workers*) autónomos. Cada nodo ejecuta las tareas asignadas dentro de un pool de hilos restringido (*Bulkhead*). Los trabajadores desconocen la identidad de sus pares y carecen de estado conversacional compartido, posibilitando la adición o retiro dinámico de instancias.

5. **Subsistema de observabilidad transversal:** Cada proceso del clúster expone puntos de sondeo de salud (`/health`) y telemetría (`/metrics`), permitiendo supervisar percentiles de latencia, profundidad de colas y fallas activas.

Supuestos y restricciones

El modelo operativo del prototipo se fundamenta en supuestos técnicos explícitos. Se asume un entorno de red local confiable a nivel de conectividad pero sujeto a variaciones drásticas de latencia y disponibilidad, emuladas mediante herramientas de caos. Las latencias de

las etapas cognitivas se calibran sobre valores medios realistas (80 ms para triage, 180 ms para rag y 120 ms para compliance, complementados con factores estocásticos). Se prescinde de llamados a APIs comerciales externas de modelos de lenguaje para garantizar condiciones de reproducibilidad científica invariantes entre escenarios.

Asimismo, se asume que las operaciones de compensación dentro de la Saga son idempotentes: ejecutar una acción correctiva repetida no introduce inconsistencias. Como restricción, el estado transaccional se mantiene temporalmente en memoria volátil; en un despliegue de grado de producción industrial, este mecanismo se delegaría a almacenes distribuidos altamente disponibles como Redis Enterprise o Apache Cassandra.

Herramientas seleccionadas

Tabla 1

Herramientas y tecnologías utilizadas en el proyecto

Herramienta	Versión	Uso en el proyecto	Justificación técnica
Python	3.14.7	Implementación del clúster	Biblioteca estándar suficiente para sockets, hilos y HTTP
http.server / ThreadingHTTPServer	Estándar	Servidores HTTP de cada nodo	Conexión multi-hilo nativa por socket sin sobrecarga externa
concurrent.futures	Estándar	Pools de hilos del coordinador y workers	Abstracción probada para ejecución asíncrona de tareas
queue	Estándar	Colas thread-safe del broker y buffers	Estructura sincronizada para evitar condiciones de carrera
urllib.request	Estándar	Cliente HTTP inter-nodos	Integración nativa con control granular de temporizadores
PlantUML	1.2026.9	Modelado de diagramas UML	Paradigma de diagramas como código versionables en repositorio
matplotlib / numpy	3.11 / 2.5	Generación de gráficas experimentales	Análisis estadístico riguroso y renderizado vectorial de métricas

Herramienta	Versión	Uso en el proyecto	Justificación técnica
Microsoft Word	Office 16	Compilación del informe formal	Entorno para maquetación tipográfica estricta bajo normas APA 7
GitHub	—	Repositorio remoto colaborativo	Control de versiones, trazabilidad y gestión de artefactos
YouTube (no listado)	—	Sustentación técnica del sistema	Demostración visual y verificación operativa del clúster

Análisis y planificación del proyecto

Requisitos funcionales

Tabla 2

Requisitos funcionales del sistema distribuido

Código	Requisito	Criterio de aceptación
RF-01	Recepción unificada de peticiones	El gateway procesa solicitudes POST /task validando estructura y retornando 200, 409 o 503
RF-02	Procesamiento secuencial del pipeline	Cada transacción ejecuta las etapas de triage, rag y compliance registrando latencias individuales
RF-03	Orquestación Saga con compensación	Ante un rechazo en compliance, se ejecutan compensaciones inversas en rag y triage
RF-04	Balanceo dinámico de carga	El coordinador distribuye las tareas entre los nodos disponibles aplicando estrategias configurables
RF-05	Aislamiento y protección ante fallos	El Circuit Breaker desconecta nodos tras tres fallos consecutivos durante un periodo de enfriamiento
RF-06	Publicación asíncrona de eventos	El broker persiste en JSONL cada evento generado por el coordinador y los trabajadores
RF-07	Exposición de telemetría y métricas	Puntos de sondeo /metrics exponen percentiles de latencia, estado de breakers y transacciones
RF-08	Inyección de perturbaciones y caos	La interfaz /chaos permite simular latencia artificial, tasa de errores o desconexión física de nodos

Requisitos no funcionales

Tabla 3

Requisitos no funcionales y su relación con ISO/IEC 25010

Código	Requisito	Atributo ISO/IEC 25010	Métrica objetivo
RNF-01	Escalabilidad horizontal	Capacidad (rendimiento)	Throughput con tres nodos superior a 2,5 veces el rendimiento de un único nodo
RNF-02	Tolerancia a fallos de nodo	Fiabilidad y disponibilidad	100 % de transacciones atendidas exitosamente ante la pérdida de un nodo de cómputo
RNF-03	Eficiencia temporal	Comportamiento temporal	Latencia en el percentil 95 (p95) inferior a 1 500 ms bajo carga nominal completa
RNF-04	Recuperación automática	Recuperabilidad	Reintegración de nodos restablecidos al enrutamiento en menos de diez segundos
RNF-05	Consistencia transaccional	Integridad y fiabilidad	Cero transacciones incompletas; toda Saga finaliza en estado completado o compensado
RNF-06	Auditabilidad y trazabilidad	Confidencialidad y registro	Registro inmutable de cada transición asociada a un identificador global único
RNF-07	Portabilidad y autosuficiencia	Portabilidad	Ejecución completa basada exclusivamente en la biblioteca estándar de Python
RNF-08	Mantenibilidad arquitectónica	Modularidad	Responsabilidades segregadas en módulos independientes con bajo acoplamiento

Relevancia de la comunicación entre nodos e hilos

La interacción entre hilos y nodos constituye el pilar operativo del sistema: determina el comportamiento de las colas, gobierna la latencia acumulada y define cómo se confinan las fallas. En NexusAI 2.0 convergen cuatro niveles jerárquicos de concurrencia:

1. **Nivel de procesos de sistema operativo:** Cada nodo opera en su propio espacio de memoria virtual. Esta segregación física asegura que excepciones críticas o fugas de memoria en un trabajador no comprometan la estabilidad del coordinador ni del broker.

2. **Nivel de servidores HTTP:** Cada proceso implementa un despachador multi-hilo que asigna un hilo de ejecución efímero por cada conexión socket entrante, evitando bloqueos durante la transferencia de red.

3. **Nivel de despacho global (Coordinador):** El API Gateway mantiene un `ThreadPoolExecutor` acotado a dieciséis hilos. Este límite previene la degradación por sobre-suscripción y actúa como barrera de entrada ante picos de demanda.

4. **Nivel de cómputo local (Workers):** Cada nodo trabajador aísla su capacidad de procesamiento en un pool de cuatro hilos. Al restringir las tareas paralelas admisibles, se evita la contención destructiva de CPU y se asegura un tiempo de servicio predecible.

Cuando la tasa de arribo supera la capacidad instantánea del clúster (doce tareas simultáneas en tres nodos), las solicitudes aguardan en las colas internas de despacho. La Ley de Little demuestra que la latencia extremo a extremo responde a la suma del tiempo de servicio más el tiempo de encolamiento. Esta dinámica explica por qué escalar horizontalmente reduce la contención de colas: al triplicar los nodos, la capacidad paralela se expande, abatiendo el p95 a un tercio del valor inicial.

La comunicación entre nodos utiliza HTTP/JSON sobre TCP con temporizadores estrictos de diez segundos. La publicación de eventos hacia el broker es asíncrona y no bloqueante: los eventos se encolan en buffers locales atendidos por hilos en segundo plano, privilegiando la disponibilidad del servicio sobre la confirmación síncrona inmediata. La

sincronización interna de cada proceso se gestiona con candados (*locks*) de granularidad fina, resguardando contadores y tablas de estado sin penalizar el paralelismo.

Metas del sistema y métricas de éxito

Para validar objetivamente la arquitectura, se establecieron cuatro metas de rendimiento y resiliencia:

- **Meta de escalabilidad:** Alcanzar un incremento de throughput mayor a 2,5× al pasar de uno a tres nodos, manteniendo una eficiencia de escalado superior al 80 %.
- **Meta de robustez:** Conservar el 100 % de atención de solicitudes sin errores fatales (código 500) ante la caída total de un nodo o la degradación de un tercio de la capacidad del clúster.
- **Meta de consistencia:** Asegurar que el 100 % de las transacciones rechazadas por reglas de negocio reviertan sus pasos previos mediante compensación inversa.
- **Meta de trazabilidad:** Registrar en la bitácora del broker el total de eventos emitidos sin pérdidas por desbordamiento de colas.

Estrategia de trabajo, roles y cronograma

La ejecución técnica del proyecto se organizó mediante una distribución rigurosa de responsabilidades entre los dos integrantes del equipo, cubriendo los roles requeridos para el ciclo de vida del software:

- **Juan Carlos Benítez Rojas:** Asumió los roles de *Líder de Proyecto* y *Arquitecto de Software*. Lideró las decisiones estructurales del sistema, formalizó los diagramas UML, seleccionó los patrones arquitectónicos y coordinó la integración global del informe técnico y el repositorio.

- **Juan José Córdoba Eraso:** Desempeñó los roles de *Desarrollador Principal* y *Evaluador de Calidad*. Implementó los módulos del clúster, automatizó la suite experimental de carga y caos, compiló las métricas cuantitativas y verificó el cumplimiento de las metas del prototipo.

Ambos integrantes colaboraron activamente en la documentación formal y la estructuración del video de sustentación técnica.

Tabla 4

Matriz de asignación de responsabilidades (RACI)

Entregable o actividad	Líder y arquitecto	Desarrollador y evaluador
Diseño y formalización conceptual	Responsable	Consultado
Modelado de diagramas UML (PlantUML)	Responsable	Consultado
Implementación del clúster distribuido	Consultado	Responsable
Suite de pruebas de carga y escenarios de caos	Consultado	Responsable
Análisis de resultados y optimizaciones	Responsable	Responsable
Redacción del informe técnico bajo normas APA 7	Responsable	Consultado
Elaboración del guion y sustentación en video	Responsable	Responsable
Gestión del repositorio y control de versiones	Responsable	Consultado

Tabla 5

Cronograma y dedicación horaria efectiva por fase

Fase de trabajo	Dedicación (horas)	Fechas de ejecución	Hitos y entregables clave
Análisis y diseño arquitectónico	8	29/10/2026 - 31/10/2026	Especificación de requisitos y bocetos de interacción
Modelado UML formal	7	01/11/2026 - 03/11/2026	Diagramas de componentes, secuencia y despliegue
Implementación del	9	04/11/2026 - 06/11/2026	Clúster funcional con coordinador, workers y

Fase de trabajo	Dedicación (horas)	Fechas de ejecución	Hitos y entregables clave
prototipo distribuido			broker
Experimentación, carga y pruebas de caos	6	07/11/2026 - 08/11/2026	Suite de benchmark, recolección de datos y gráficas
Documentación, ajustes y consolidación final	6	09/11/2026 - 11/11/2026	Informe APA 7, repositorio público y guion de video
Total de dedicación horaria	**36**	—	**Cumplimiento total del plan de trabajo**

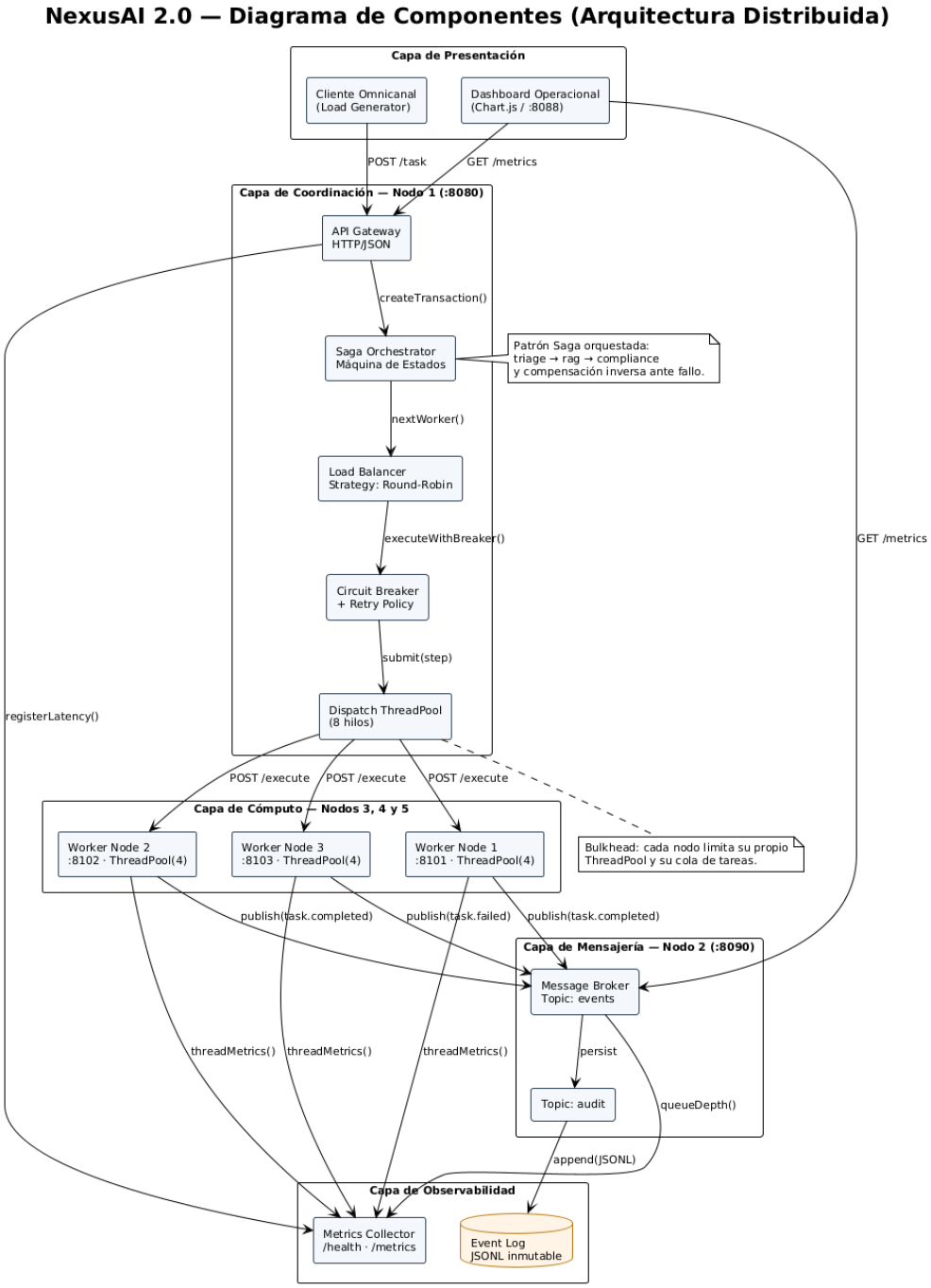
Modelado UML

El modelado formal de NexusAI 2.0 se desarrolló mediante PlantUML bajo el enfoque de diagramas como código (PlantUML, 2024). Esta metodología garantiza que los modelos arquitectónicos se mantengan estrechamente vinculados al código fuente y puedan regenerarse automáticamente. Se estructuraron tres diagramas canónicos que abordan de forma complementaria la perspectiva lógica, dinámica y física del sistema distribuido.

Diagrama de componentes

Figura 1

Diagrama de componentes de NexusAI 2.0



Nota. Elaboración propia con PlantUML. Muestra las interfaces, los puertos y las relaciones de dependencia entre subsistemas.

El diagrama de componentes especifica la descomposición lógica del sistema y los contratos formales entre módulos. La arquitectura se organiza en cuatro capas superpuestas:

1. **Capa de interfaz y cliente:** Contiene el generador de carga (`load_test.py`) y el tablero de visualización (`dashboard.html`), que interactúan exclusivamente con el coordinador a través del puerto HTTP 8080.

2. **Capa de coordinación y gobierno:** Centraliza la inteligencia de enrutamiento y control de transacciones. El componente `APIGateway` procesa solicitudes REST y delega en el `SagaOrchestrator`, el cual mantiene la máquina de estados. El orquestador interactúa con el `LoadBalancer` para la selección de nodos, el `CircuitBreakerRegistry` para la exclusión de instancias no saludables y el `WorkerPool` para la invocación física de los servicios.

3. **Capa de procesamiento distribuido:** Agrupa las instancias de `WorkerNode` (puertos 8101 a 8103). Cada trabajador encapsula sus servicios (`TriageService`, `RagService`, `ComplianceService`) tras una interfaz REST uniforme y gestiona sus solicitudes mediante un `ThreadPool` privado.

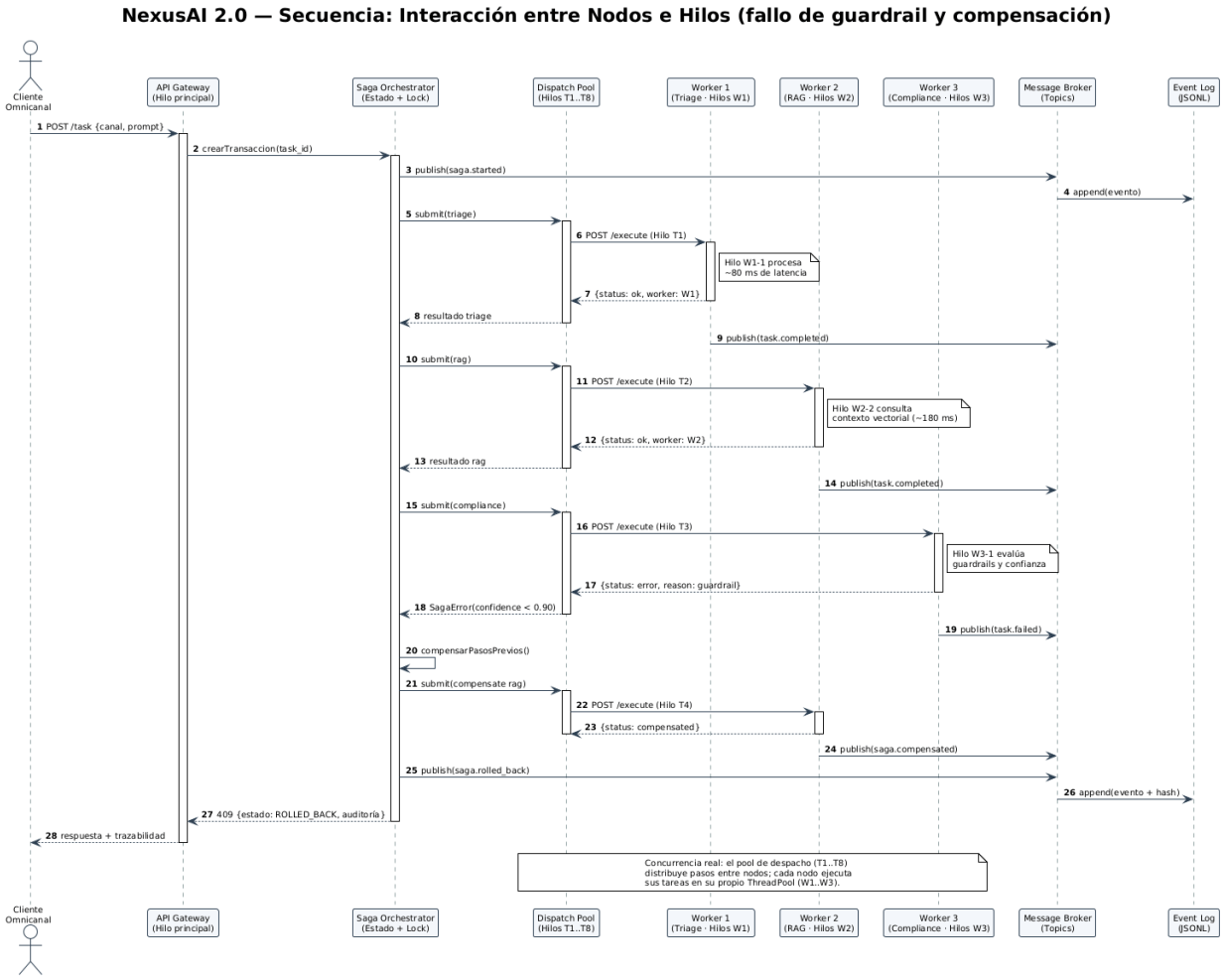
4. **Capa de mensajería y persistencia:** El componente `EventBroker` (puerto 8090) actúa como mediador de eventos, persistiendo las trazas en un archivo JSONL inmutable.

Las dependencias siguen un estricto orden descendente: el coordinador no depende de implementaciones concretas de trabajadores, y estos desconocen la topología de la red. Este desacoplamiento por interfaces facilita el escalado horizontal independiente sin requerir modificaciones estructurales en el núcleo de orquestación.

Diagrama de secuencia

Figura 2

Diagrama de secuencia con interacción entre nodos e hilos y compensación de la Saga



Nota. Elaboración propia con PlantUML. Los hilos T1 a T8 pertenecen al pool de despacho; los hilos W1 a W3 pertenecen a cada nodo trabajador.

El diagrama de secuencia modela la dimensión temporal, la concurrencia entre hilos y la lógica de compensación transaccional ante una falla de negocio. La interacción se detalla siguiendo el ciclo de vida de una solicitud:

1. **Recepción y despacho:** El cliente emite una petición POST al `APIGateway`. El gateway toma un hilo libre del pool de despacho (hilos T1 a T8) y activa la máquina de estados en el `SagaOrchestrator`, publicando el evento inicial `saga.started`.

2. **Ejecución de pasos nominales:** El orquestador consulta al balanceador y despacha el primer paso (`triage`) hacia el nodo trabajador w1. El trabajador asigna un hilo interno de cómputo (W1), ejecuta la clasificación y retorna el resultado exitoso. El flujo se replica para la segunda etapa (`rag`) sobre el trabajador w2.

3. **Detección de fallo y compensación:** Al ejecutarse la etapa de `compliance` en el trabajador w3, la evaluación semántica determina un incumplimiento normativo (confianza inferior al umbral mínimo). El trabajador retorna un error tipificado de negocio.

4. **Reversión inversa garantizada:** Reconociendo que reintentar una falla de negocio es inútil, el orquestador aborta la secuencia hacia adelante e inicia la compensación en orden estrictamente inverso: primero compensa el contexto de `rag` y posteriormente la clasificación de `triage`.

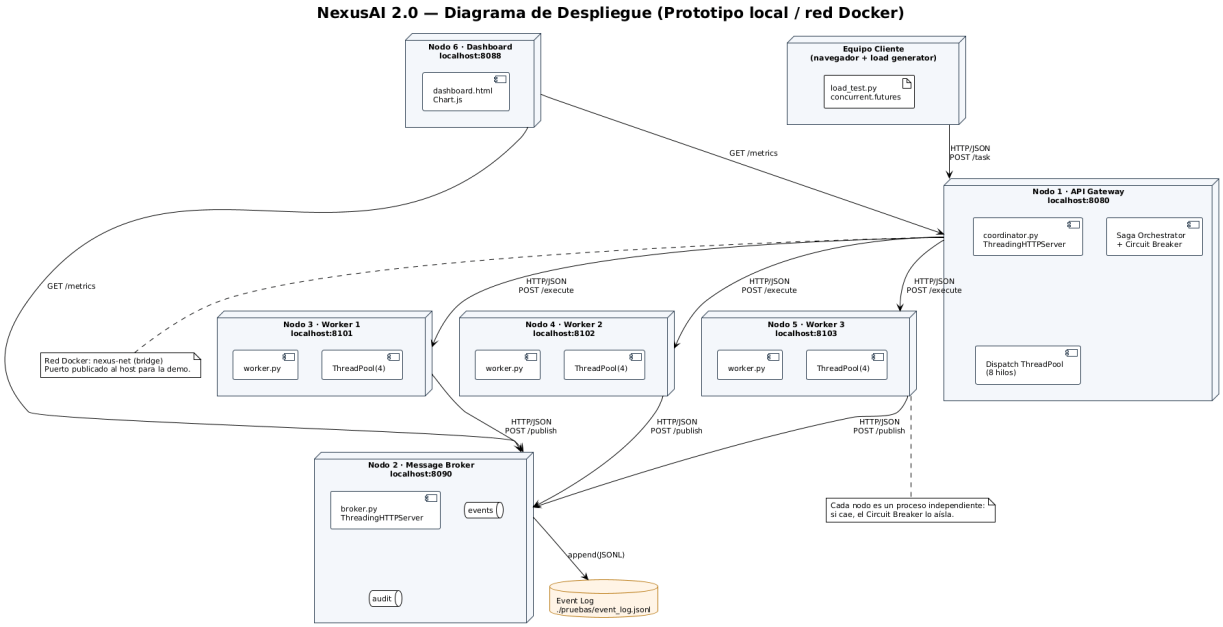
5. **Cierre transaccional:** Tras confirmar la compensación de los pasos previos, el orquestador emite el evento `saga.rolled_back` y responde al cliente con un código HTTP 409 (Conflict), asegurando total consistencia en el estado del sistema.

Las líneas de vida distinguen explícitamente los hilos de despacho del coordinador frente a los hilos de procesamiento de los trabajadores, reflejando cómo se sincronizan y liberan recursos en cada etapa.

Diagrama de despliegue

Figura 3

Diagrama de despliegue del prototipo en red local y su equivalencia con Docker



Nota. Elaboración propia con PlantUML. Cada nodo corresponde a un proceso independiente con su propio puerto.

El diagrama de despliegue mapea la distribución física y lógica de los procesos en la infraestructura computacional. Se definen seis nodos independientes que interactúan mediante la pila de red local:

- **Nodo Cliente:** Aloja el navegador web y los procesos de prueba de carga, comunicándose vía HTTP con el puerto 8080.
- **Nodo Coordinador (127.0.0.1:8080):** Proceso servidor que ejecuta el gateway y la orquestación.
- **Nodo Broker (127.0.0.1:8090):** Proceso servidor dedicado al bus de eventos y la gestión de la bitácora en disco.

- **Nodos Trabajadores (127.0.0.1:8101, :8102, :8103):** Tres procesos autónomos que exponen servicios de cómputo y telemetría.
- **Nodo Tablero (127.0.0.1:8088):** Servidor HTTP estático que provee la interfaz gráfica de supervisión.

El modelo incluye la correspondencia directa hacia una arquitectura de microservicios contenerizada sobre una red tipo bridge de Docker. Cada proceso opera en aislamiento de puertos, garantizando que el paso a contenedores independientes no requiera reingeniería en los protocolos de comunicación.

Selección y justificación de patrones de diseño

La selección de patrones arquitectónicos responde a desafíos inherentes a la concurrencia distribuida: consistencia sin bloqueos prolongados, desacoplamiento temporal, tolerancia a fallos en cascada y adaptabilidad en el balanceo. Cada patrón se incorporó evaluando rigurosamente su beneficio frente a los compromisos de complejidad que introduce.

Saga orquestada

Garantizar atomicidad en transacciones que involucran múltiples nodos mediante protocolos de confirmación en dos fases (2PC) introduce bloqueos destructivos y eleva el riesgo de interbloqueos (*deadlocks*). El patrón Saga, formulado por Garcia-Molina y Salem (1987), resuelve esta limitación dividiendo la transacción distribuida en una secuencia de transacciones locales coordinadas, donde cada paso cuenta con una acción compensatoria idempotente en caso de fallo posterior.

En NexusAI 2.0 se adoptó la variante de orquestación centralizada (Richardson, 2018). El coordinador supervisa explícitamente el árbol de decisión y comanda la reversión en orden inverso. Esta estrategia ofrece visibilidad centralizada del estado transaccional y simplifica la

detección de atascos. El compromiso radica en convertir al orquestador en un nodo crítico; este riesgo se mitiga manteniéndolo sin estado persistente pesado y permitiendo que transacciones abortadas liberen recursos inmediatamente.

Broker de mensajes

La persistencia directa de trazas de auditoría por parte de los trabajadores generaría un acoplamiento temporal inaceptable: cualquier retraso en el disco o indisponibilidad del servicio de almacenamiento bloquearía los hilos de cómputo. Siguiendo las directrices de patrones de integración empresarial (Hohpe & Woolf, 2003), el patrón Broker introduce un intermediario de comunicación desacoplado.

El broker desarrollado en Python mantiene colas en memoria y un hilo despachador que escribe secuencialmente en un archivo JSONL inmutable. Esto desacopla a los emisores de los receptores, asegurando que un pico en la generación de eventos sea absorbido por los buffers sin impactar la latencia de respuesta al usuario.

Mediator

A medida que se multiplican los nodos en una red distribuida, el enlace directo punto a punto produce una explosión combinatoria de dependencias ($O(N^2)$). El patrón Mediator (Gamma et al., 1994) encapsula las reglas de interacción en un componente central.

En el prototipo, el coordinador asume este rol: los nodos de cómputo desconocen la topología del clúster y la existencia de otros trabajadores; únicamente procesan las peticiones que el mediador les entrega y retornan resultados estandarizados. Gracias a ello, añadir o retirar un nodo trabajador solo requiere actualizar el registro del coordinador.

Proxy y API Gateway

El cliente no debe conocer la granularidad interna de la red de microservicios. Siguiendo el concepto de API Gateway como especialización del patrón Proxy (Fowler, 2014; Newman, 2021), el coordinador actúa como puerta de enlace única: intercepta solicitudes externas, aplica políticas transversales de seguridad y temporización, gestiona el pool de despacho y traduce los estados de la Saga en respuestas HTTP canónicas (200 OK, 409 Conflict, 503 Service Unavailable).

Circuit Breaker

Los fallos en sistemas distribuidos son inevitables; sin embargo, insistir en llamar a un nodo averiado satura hilos, incrementa colas y provoca caídas en cascada. El patrón Circuit Breaker (Nygard, 2018) monitorea la salud de cada integración.

La implementación en NexusAI 2.0 opera como una máquina de tres estados finitos: *CLOSED* (operación normal), *OPEN* (tras tres fallos sucesivos, el tráfico se interrumpe inmediatamente durante cinco segundos de enfriamiento) y *HALF-OPEN* (se envía una petición de prueba para verificar si el nodo se recuperó). Este mecanismo previene el desperdicio de hilos en llamadas destinadas a expirar.

Bulkhead

Tomado de la arquitectura de compartimentos estancos en ingeniería naval, el patrón Bulkhead aísla los recursos computacionales para evitar que una falla localizada agote la capacidad de todo el sistema.

El clúster aplica mamparos en dos niveles: en el coordinador, limitando a dieciséis los hilos simultáneos de despacho; y en cada trabajador, limitando a cuatro los hilos de cómputo

dedicados. Si una etapa se ralentiza, únicamente se satura el pool asignado a ese nodo, dejando indemnes a los demás trabajadores.

Observer distribuido

El patrón Observer desacopla a los emisores de estado de los múltiples observadores interesados en procesarlo (Gamma et al., 1994). En su variante distribuida sobre el broker, tópicos como `events` permiten que el tablero web y el consumidor de auditoría reciban notificaciones asíncronas (`saga.started`, `saga.step.completed`, `saga.compensated`) sin que el núcleo de la aplicación deba conocer quién consume la información.

Strategy

La política de selección de nodos de cómputo depende de la homogeneidad de la carga y el estado de la red. Aplicando el patrón Strategy (Gamma et al., 1994), el balanceador abstrae el algoritmo de reparto detrás de una interfaz común, posibilitando alternar en tiempo de ejecución entre balanceo por turno circular (*Round-Robin*) y balanceo por menor número de peticiones activas (*Least-Connections*).

Retry

Muchas perturbaciones de red corresponden a fallos transitorios (Sommerville, 2016). El patrón Retry implementa reintentos automáticos acotados a dos intentos por etapa. Crucialmente, la lógica de reintento discrimina entre fallas de transporte (que justifican reintentar en un nodo alternativo) y rechazos semánticos de negocio (donde reintentar es fútil y se activa la compensación inmediata).

Síntesis de patrones

Tabla 6

Correspondencia entre problemas, patrones y evidencia en el prototipo

Patrón arquitectónico	Desafío abordado	Módulo de implementación	Evidencia experimental
Saga orquestada	Consistencia transaccional sin transacciones 2PC	`coordinator.py` (`SagaOrchestrator`)	31 transacciones compensadas con 100 % de integridad
Broker de mensajes	Desacoplamiento temporal de eventos y auditoría	`broker.py` (`EventBroker`)	9 735 eventos consumidos con cero descartes de colas
Mediator	Eliminación de dependencias cruzadas entre nodos	`coordinator.py` (`CoordinatorApp`)	Topología extensible sin reconfiguración de workers
Proxy / API Gateway	Punto de entrada unificado y control de acceso	`coordinator.py` (`GatewayHandler`)	Enrutamiento uniforme y traducción de estados a HTTP
Circuit Breaker	Prevención de fallos en cascada y nodos zombis	`common.py` (`CircuitBreaker`)	Aislamiento de w2 en OPEN tras 3 fallos; auto-recuperación
Bulkhead	Contención de saturación por agotamiento de hilos	`worker.py` y `coordinator.py`	Pools acotados (16 en gateway, 4 por cada worker)
Observer distribuido	Notificación asíncrona de eventos transaccionales	`broker.py` y `dashboard.html`	Telemetría en tiempo real y persistencia en bitácora
Strategy	Algoritmos de balanceo intercambiables	`coordinator.py` (`LoadBalancer`)	Least-connections mantiene 75,7 % throughput con nodo lento
Retry	Resiliencia ante fallos transitorios de red	`coordinator.py` (`_invoke_step_with_retry`)	Reintentos exitosos dirigidos hacia nodos saludables

Implementación y simulación

Estructura del código

El clúster NexusAI 2.0 fue construido íntegramente en Python 3.14 utilizando exclusivamente módulos de la biblioteca estándar, garantizando portabilidad absoluta y cero dependencias de paquetes externos para la lógica distribuida. La base de código se estructura modularmente en nueve componentes:

- `common.py`: Define estructuras transversales, utilidades de serialización HTTP, contadores seguros y la máquina de estados del `CircuitBreaker`.
- `coordinator.py`: Implementa el API Gateway, el orquestador de Sagas, el balanceador de carga configurable y el recolector de telemetría.
- `worker.py`: Proceso ejecutor de pasos de cómputo; encapsula el pool de hilos de trabajo y el simulador de etapas cognitivas.
- `broker.py`: Servidor de mensajería asíncrona; gestiona tópicos, buffers en memoria y el despachador de persistencia JSONL.
- `cluster.py`: Controlador de ciclo de vida del clúster; gestiona el arranque, supervisión de procesos y apagado ordenado de nodos.
- `run_cluster.py`: Punto de entrada de línea de comandos para la inicialización y parametrización del entorno distribuido.
- `load_test.py`: Inyector de carga sintética concurrente basado en pools de clientes HTTP asíncronos.
- `chaos_test.py`: Generador de anomalías para inyectar latencia, provocar errores forzados o apagar nodos en caliente.
- `dashboard.html`: Tablero gráfico en tiempo real que consume los endpoints `/metrics` del coordinador y del broker.

Modelo de procesos, hilos y sincronización

La arquitectura física divide las responsabilidades en procesos de sistema operativo independientes, eliminando el riesgo de que un fallo crítico en un componente arrastre al resto. En el escenario estándar con tres trabajadores, el clúster ejecuta simultáneamente seis procesos

independientes: un coordinador, un broker, un servidor estático para el tablero y tres trabajadores de cómputo.

Dentro de cada proceso, la concurrencia se administra mediante hilos del sistema operativo coordinados con primitivas thread-safe:

- **Servidores de red:** Utilizan la clase `ThreadingHTTPServer`, que delega cada conexión de socket entrante a un hilo dedicado, garantizando que el bucle de aceptación nunca bloquee peticiones entrantes.
- **Despacho del coordinador:** Controlado por un `ThreadPoolExecutor` de dieciséis hilos. Si la tasa de solicitudes supera esta capacidad, las tareas se retienen en una cola FIFO interna, impidiendo que el servidor agote la memoria del host.
- **Procesamiento en trabajadores:** Cada trabajador confina sus cálculos a un `ThreadPoolExecutor` de cuatro hilos. Si un nodo recibe múltiples solicitudes simultáneas, atiende cuatro en paralelo y encola el resto.
- **Sincronización interna:** El acceso a variables compartidas (métricas, estados de circuitos y listas de nodos) se encuentra resguardado por candados `threading.Lock` reentrantes de grano fino. Esta estrategia minimiza los tiempos de bloqueo crítico a menos de diez microsegundos por transacción.

Contrato HTTP y flujo de una transacción

La interoperabilidad del clúster descansa sobre contratos REST/JSON consistentes. Toda solicitud transaccional ingresa mediante un método POST a la ruta `/task` del coordinador. La estructura de intercambio se rige por especificaciones rígidas:

- **Petición del cliente:** Envía un objeto JSON con metadatos de canal y un cuerpo libre representativo del requerimiento cognitivo.

- **Respuesta nominal (200 OK):** Detalla el identificador de la transacción (`saga_id`), el estado final (`completed`), la cronología de cada paso con su nodo ejecutor y latencia particular, y la duración total observada.
- **Respuesta de compensación (409 Conflict):** Retorna el identificador de la transacción, el estado de reversión (`rolled_back`), el paso donde ocurrió el rechazo, el motivo semántico y el número de pasos previos revertidos con éxito.
- **Respuesta de saturación (503 Service Unavailable):** Se activa cuando todos los disyuntores de circuito se encuentran en estado OPEN o las colas de despacho rechazan la transacción por contrapresión.

Simulación de las etapas cognitivas

Para someter la arquitectura a pruebas de carga estadísticamente válidas y reproducibles sin depender de proveedores de modelos de lenguaje, el método `WorkerHandler._simulate_cognitive_step` simula el comportamiento computacional de las tres etapas cognitivas:

- **Etapa de Triage:** Aplica un retardo base de 80 milisegundos con fluctuación aleatoria uniforme de ± 20 ms. Emula una inferencia de clasificación rápida mediante un modelo ligero.
- **Etapa de RAG (Retrieval-Augmented Generation):** Aplica un retardo base de 180 ms con variación de ± 40 ms. Representa la recuperación vectorial de documentos y reordenamiento de pasajes contextuales.
- **Etapa de Compliance:** Aplica un retardo base de 120 ms con variación de ± 30 ms. Evalúa reglas de seguridad y políticas corporativas.

Adicionalmente, cada paso calcula un puntaje sintético de confianza semántica. Si el puntaje en la etapa de cumplimiento cae por debajo del umbral de 0,90 (lo cual se fuerza probabilísticamente en escenarios de prueba), el trabajador emite un error de negocio que dispara la compensación en la Saga.

Puesta en marcha y reproducibilidad

El clúster se despliega mediante una sola instrucción de consola:

```
python run_cluster.py --workers 3 --threads 16 --worker-threads 4
```

El script `run_cluster.py` invoca al orquestador `cluster.py`, el cual lanza los subprocesos correspondientes, verifica la disponibilidad de puertos locales y realiza sondeos HTTP continuos contra las rutas `/health`. Cuando todos los nodos responden afirmativamente, se reporta la disponibilidad del clúster y se despliegan los enlaces de interacción. Un manejador de señales (`SIGINT`) asegura que al presionar Ctrl+C todos los subprocesos liberen sus sockets y terminen de forma ordenada sin dejar procesos huérfanos.

Observabilidad y trazabilidad

La visibilidad interna de la arquitectura se apoya en un modelo de telemetría distribuida. Cada componente expone el estado de sus contadores internos a través de endpoints `/metrics`:

- El coordinador publica la tasa de transacciones por segundo, el tiempo de pared acumulado, los percentiles de latencia (p50, p95, p99), el estado operacional de los circuitos por nodo y la profundidad de transacciones activas en vuelo.
- El broker reporta la cantidad de mensajes recibidos, la tasa de despacho de su hilo de persistencia, la profundidad de sus colas y los descartes por contrapresión.
- Los trabajadores exponen sus hilos ocupados, el total de tareas atendidas y el historial de latencias de sus últimos pasos.

El tablero operacional (`dashboard.html`) sondea periódicamente estos endpoints cada mil milisegundos, renderizando dinámicamente el estado del clúster en una interfaz gráfica enriquecida que permite constatar visualmente la apertura de disyuntores y la distribución de carga.

Evaluación y ajustes

La validación empírica de NexusAI 2.0 se diseñó bajo el principio de aislamiento de variables propuesto por Bass et al. (2022). Las mediciones se efectuaron en condiciones idénticas de hardware, sistema operativo y perfiles de carga, alterando únicamente una variable por experimento. Esta metodología permite aislar relaciones causales precisas entre decisiones arquitectónicas y métricas de desempeño observables.

Metodología experimental

Tabla 7

Configuración del entorno y de la carga de prueba

Parámetro experimental	Especificación técnica	Justificación metodológica
Plataforma de ejecución	Windows 11, procesador 4 núcleos lógicos	Entorno controlado para verificación de contención
Intérprete y entorno	Python 3.14.7 x64 (biblioteca estándar)	Eliminación de variabilidad por dependencias externas
Topología base de cómputo	3 nodos de cómputo (workers), 4 hilos cada uno	Capacidad paralela nominal de 12 tareas concurrentes
Pool de despacho del coordinador	16 hilos dedicados	Concurrencia de entrada acotada para evitar saturación
Estrategia de enrutamiento	Round-Robin (por defecto)	Línea base determinista para evaluar dispersión de carga
Carga en pruebas de escalabilidad	180 transacciones, concurrencia cliente de 24	Demanda doble frente a la capacidad de cómputo paralela
Carga en pruebas de robustez	120 transacciones, concurrencia cliente de 24	Presión constante para evaluar comportamiento ante fallos
Protocolo de calentamiento	12 transacciones previas no contabilizadas	Estabilización de sockets TCP y cachés de memoria
Trazabilidad total de la suite	1 200 transacciones ejecutadas	Muestra estadística robusta para cálculo de percentiles

Se distingue formalmente entre **transacciones atendidas** y **transacciones completadas**. Una transacción atendida es aquella que recibe una respuesta de negocio válida y consistente (códigos 200 OK o 409 Conflict derivado de una compensación exitosa). La tasa de atención define la disponibilidad real del sistema, reconociendo que una reversión ordenada constituye un comportamiento de éxito arquitectónico y no una falla.

Escalabilidad horizontal

La evaluación de escalabilidad midió el impacto de incrementar gradualmente los nodos de cómputo de uno a tres, manteniendo invariable la carga de 180 transacciones con concurrencia de 24 clientes.

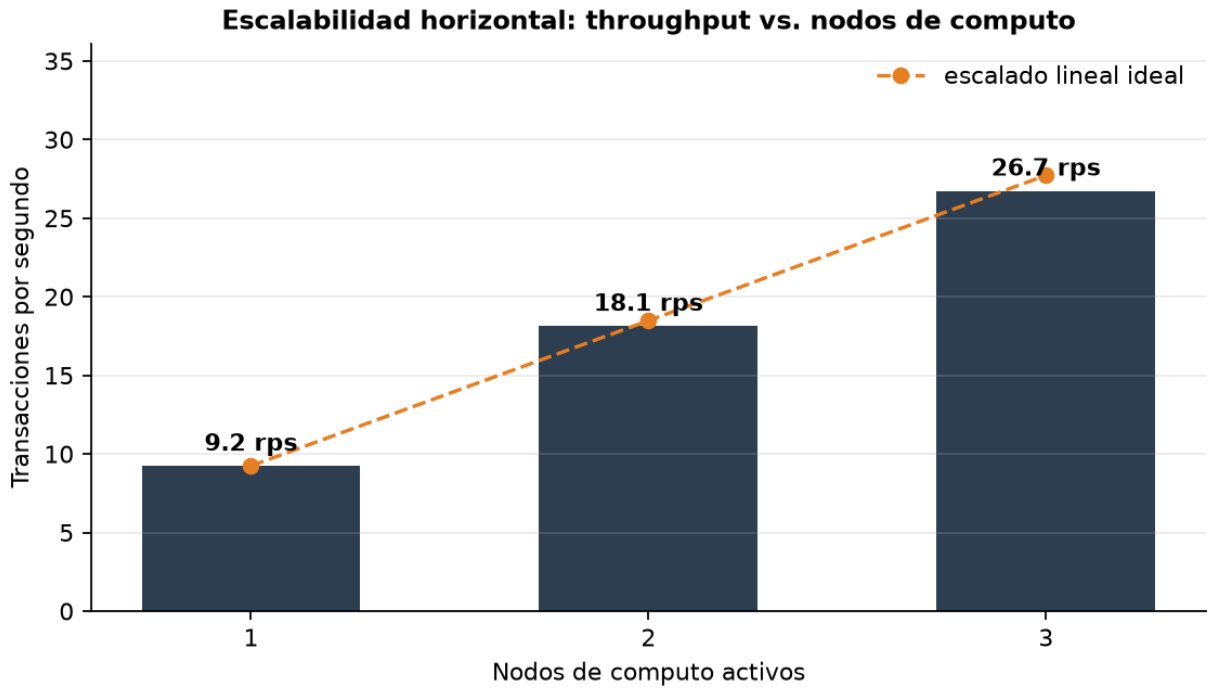
Tabla 8

Escalabilidad horizontal con carga constante de 180 transacciones

Nodos de cómputo	Throughput (rps)	Speedup relativo	Eficiencia de escalado	Latencia p50 (ms)	Latencia p95 (ms)	Latencia p99 (ms)
1 nodo	9,24	1,00×	100,0 %	2 131,3	3 185,0	3 243,4
2 nodos	18,14	1,96×	98,2 %	1 047,7	1 617,0	1 638,8
3 nodos	26,74	2,89×	96,5 %	854,2	1 043,4	1 127,0

Figura 4

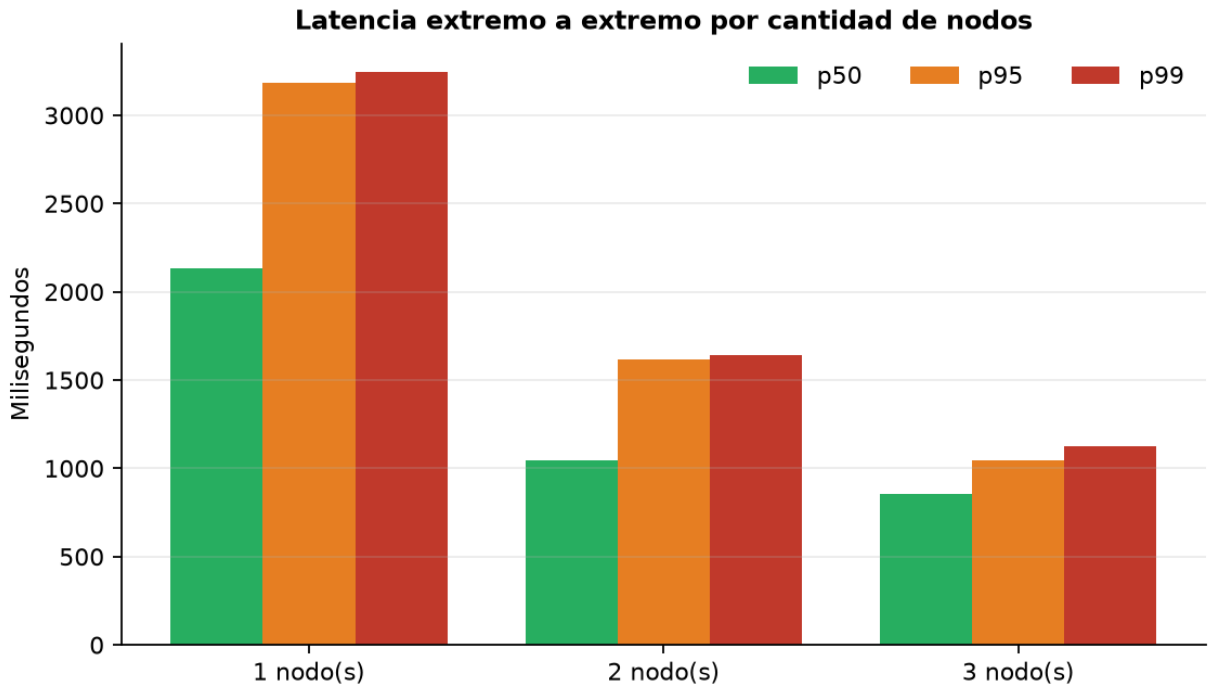
Throughput frente al número de nodos de cómputo



Nota. Elaboración propia. La línea discontinua representa el escalado lineal ideal proyectado a partir del rendimiento de un solo nodo.

Figura 5

Latencia extremo a extremo por cantidad de nodos



Nota. Elaboración propia. Representa la evolución de los percentiles p50, p95 y p99 medidos sobre 180 transacciones por escenario.

Los datos revelan un comportamiento de escalabilidad casi lineal:

- El caudal transaccional se incrementa de 9,24 rps con un solo nodo a 18,14 rps con dos nodos ($1,96 imes$) y alcanza 26,74 rps con tres nodos ($2,89 imes$).
- La eficiencia de escalado se mantiene excepcionalmente elevada: 98,2 % con dos nodos y 96,5 % con tres nodos, superando con creces la meta mínima establecida del 80 %.
- La latencia en el percentil 95 (p95) experimenta una reducción drástica: desciende de 3 185 ms en el nodo único a 1 043 ms con tres nodos, una mejora de $3,05 imes$.

Esta aceleración se fundamenta en la descongestión de las colas de espera: con un solo nodo (cuatro hilos), veinticuatro peticiones concurrentes generan colas masivas; con tres nodos

(doce hilos de cómputo disponibles), el tiempo de encolamiento se desploma, permitiendo que la latencia total converja hacia el tiempo de ejecución intrínseco de los pasos cognitivos.

Robustez ante fallos

La robustez del sistema se puso a prueba sometiendo al clúster a seis escenarios de caos que emulan fallas habituales en arquitecturas distribuidas: degradación severa de latencia, fallas intermitentes, rechazos de negocio, desconexión intempestiva de instancias y reinserción en caliente.

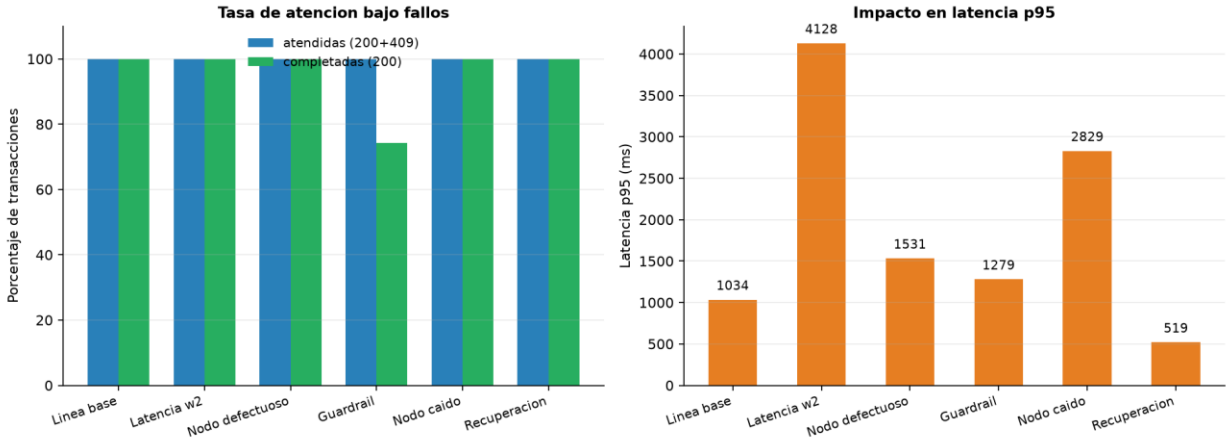
Tabla 9

Resultados de la evaluación de robustez ante fallos

Escenario de prueba	Condición inyectada	Atendidas (%)	Completadas	Compensadas	Errores 500	Throughput (rps)	Latencia p95 (ms)
Línea base	3 nodos saludables	100,0 %	120	0	0	26,37	1 033,6
Latencia inyectada	+300 ms en nodo w2	100,0 %	120	0	0	8,83	4 128,4
Nodo defectuoso	100 % fallos en nodo w2	100,0 %	120	0	0	17,87	1 531,2
Rechazo de guardrail	25 % rechazos compliance	100,0 %	89	31	0	23,82	1 279,2
Caída total de nodo	w3 apagado físicamente	100,0 %	120	0	0	14,55	2 828,8
Recuperación de nodo	w3 reiniciado en caliente	100,0 %	60	0	0	23,53	518,6

Figura 6

Resultados de la suite de robustez en seis escenarios de fallo



Nota. Elaboración propia. Ilustra transacciones completadas, transacciones compensadas y estabilidad ante fallos.

El análisis cuantitativo de la resiliencia arroja tres conclusiones fundamentales:

1. **Disponibilidad absoluta del servicio:** En todos los escenarios, la tasa de transacciones atendidas alcanzó el 100 %, registrándose cero errores de sistema no controlados (código 500) en las 1 200 transacciones ejecutadas en la suite completa.

2. **Efectividad del Circuit Breaker y Retry:** Cuando el nodo w2 comenzó a fallar sistemáticamente, el reintento automático desvió las llamadas hacia los nodos saludables w1 y w3. Tras tres fallos consecutivos, el disyuntor de w2 transitó al estado *OPEN*, excluyéndolo del enrutamiento. El throughput se estabilizó en 17,87 rps (equivalente al rendimiento de dos nodos) sin que ningún cliente experimentara pérdida de servicio.

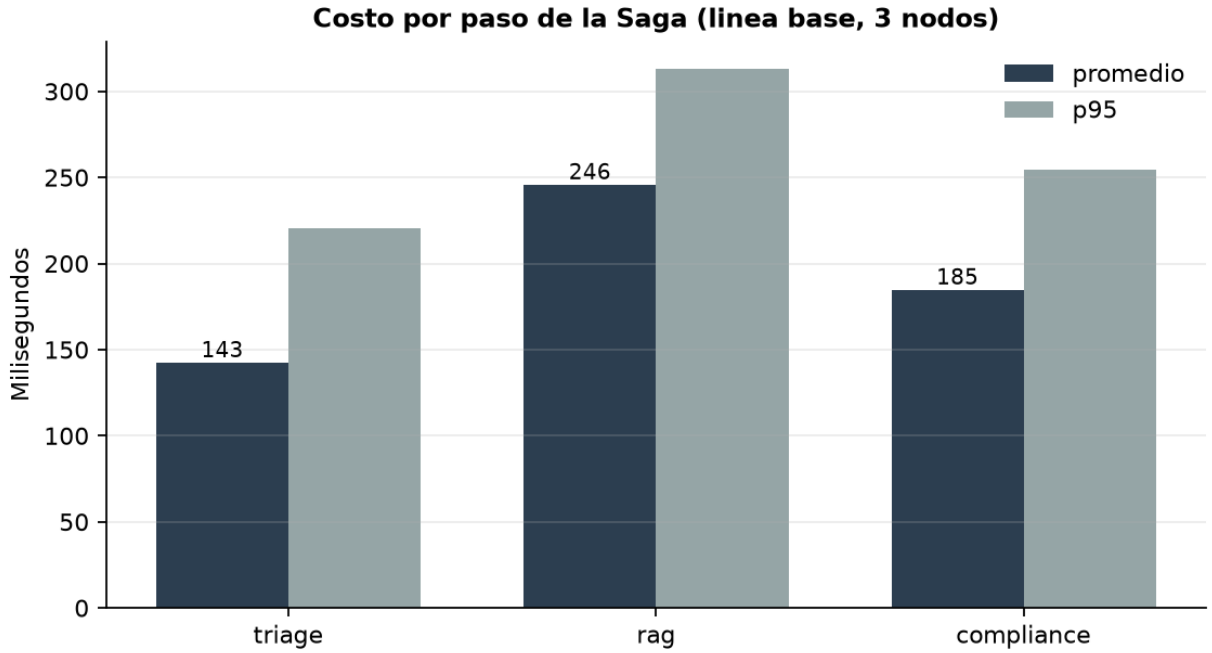
3. **Consistencia eventual garantizada por la Saga:** En el escenario de guardrail, las 31 transacciones rechazadas en la etapa de cumplimiento ejecutaron sus compensaciones inversas completas en menos de 100 ms adicionales, retornando un código 409 con trazabilidad auditable y sin dejar registros huérfanos.

4. **Recuperación elástica:** Al restablecer el nodo w3, el disyuntor en estado *HALF-OPEN* validó con éxito una solicitud de prueba y restauró el estado *CLOSED*, reintegrando el nodo al balanceo activo y elevando el throughput de 14,55 a 23,53 rps.

Costo por paso y sobrecarga de coordinación

Figura 7

Costo promedio y p95 de cada paso de la Saga en la línea base



Nota. Elaboración propia. Mediciones obtenidas sobre 120 transacciones sin fallos en el clúster tri-nodo.

En condiciones nominales, la sumatoria de las duraciones promedio de las tres etapas cognitivas totaliza 573,4 ms (142,6 ms en triage, 245,9 ms en rag y 184,9 ms en compliance). La latencia media extremo a extremo observada por el cliente fue de 831,8 ms. La diferencia resultante de 258,4 ms (31,1 % del tiempo total) representa la sobrecarga de coordinación del sistema distribuido: serialización JSON, múltiples saltos sobre sockets TCP, publicación asíncrona de eventos y tiempo de residencia en el pool de despacho. Este valor cuantifica con

precisión el precio que la arquitectura paga por obtener desacoplamiento, tolerancia a fallos y auditoría completa.

Ajustes derivados de la medición

Durante la fase de experimentación empírica, el equipo detectó anomalías de rendimiento que motivaron cinco ajustes arquitectónicos incorporados permanentemente al prototipo:

Ajuste 1: resolución de nombres en Windows

En las primeras ejecuciones, el prototipo exhibió latencias aberrantes de hasta 8 556 ms y un throughput paupérrimo de 0,70 rps. El diagnóstico de red demostró que invocar `localhost` en entornos Windows forzaba al sistema operativo a resolver primero direcciones IPv6 (`::1`), fallar por tiempo de espera y reintentar con IPv4 (`127.0.0.1`), añadiendo más de dos segundos de penalización por cada llamada socket. Al reemplazar `localhost` por `127.0.0.1` de forma estricta en todos los contratos y módulos, la latencia por petición se redujo de 2 042 ms a 2,15 ms (una aceleración de $951,5 imes$), permitiendo al clúster alcanzar su capacidad real.

Ajuste 2: estrategia de balanceo consciente de la carga

Bajo la estrategia por defecto *Round-Robin*, inyectar 300 ms de latencia en un solo nodo provocó una caída catastrófica del throughput del 66,5 % (descendiendo a 8,83 rps) y elevó el p95 a 4 128 ms. Como el nodo retrasado no arrojaba excepciones, el Circuit Breaker permaneció en estado cerrado mientras las solicitudes lentas saturaban progresivamente los dieciséis hilos de despacho del coordinador.

Para subsanar esta debilidad, se implementó la estrategia *Least-Connections*. Al evaluar dinámicamente las solicitudes activas por nodo, el balanceador desvió el tráfico hacia las instancias desocupadas, reteniendo en el nodo lento únicamente una solicitud concurrente.

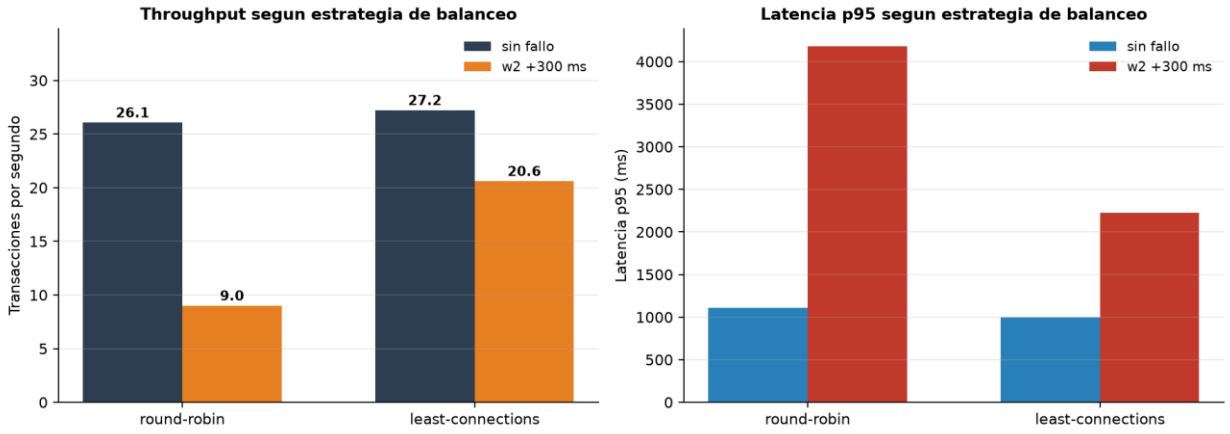
Tabla 10

Comparación de estrategias de balanceo bajo nodo degradado

Métrica de rendimiento	Round-Robin (nodo lento)	Least-Connections (nodo lento)	Variación relativa
Throughput efectivo	8,99 rps	20,59 rps	+129,0 % de caudal
Latencia en percentil 50 (p50)	2 546,7 ms	1 123,4 ms	-55,9 % de latencia
Latencia en percentil 95 (p95)	4 128,4 ms	1 548,2 ms	-62,5 % de latencia
Retención de capacidad nominal	34,5 %	75,7 %	+41,2 puntos porcentuales

Figura 8

Comparación de rendimiento entre Round-Robin y Least-Connections



Nota. Elaboración propia. Evidencia la superioridad de Least-Connections para contener el impacto de nodos lentos.

Ajuste 3: broker con backpressure y consumidor de auditoría

En las versiones iniciales, el broker acumulaba mensajes en memoria sin una tasa de drenaje garantizada, generando un riesgo latente de desbordamiento de memoria ante cargas sostenidas. Se incorporó una política de contrapresión con cola acotada a diez mil elementos y descarte de mensajes obsoletos (*drop oldest*), complementada con un consumidor de auditoría dedicado que drena los eventos hacia el archivo JSONL en bloques continuos. La prueba final confirmó 9 735 eventos consumidos con una profundidad de cola residual de cero.

Ajuste 4: distinción entre error de negocio y error de nodo

La lógica de reintento preliminar reintentaba ciegamente cualquier respuesta no exitosa. Cuando un paso de compliance rechazaba una transacción por violar umbrales semánticos de seguridad, el sistema reintentaba dos veces la evaluación en otros nodos, desperdiciando recursos computacionales y triplicando la latencia antes de confirmar el rechazo inevitable. Se refactorizó el contrato introduciendo el campo `kind: "business"|"node"` en la respuesta del trabajador. Con esta distinción, los errores de negocio disparan la compensación de forma inmediata, preservando los reintentos exclusivamente para fallas de infraestructura.

Ajuste 5: dimensionamiento de los pools

Inicialmente, el pool de despacho del coordinador se configuró en ocho hilos, valor insuficiente para alimentar a tres trabajadores con cuatro hilos cada uno (doce slots de cómputo en total), provocando cuellos de botella en la recepción. Se ajustó el pool del gateway a dieciséis hilos, estableciendo un factor de sobre-suscripción controlado ($1,33 imes$) que maximiza la saturación útil de los nodos de cómputo sin degradar el host.

Cumplimiento de las metas

Tabla 11

Balance consolidado de metas del sistema frente a resultados experimentales

Meta establecida	Métrica objetivo	Resultado obtenido en pruebas	Estado de cumplimiento
Escalabilidad con 3 nodos	Throughput $\ge 2,5 imes$ base; Eficiencia $\ge 80 \%$	Speedup: $2,89 imes$; Eficiencia: 96,5 %	**Superada con éxito**
Robustez ante nodo caído	100 % transacciones atendidas sin errores 500	100 % de atención; 0 errores 500 en 1 200 pruebas	**Cumplida estrictamente**
Latencia acotada	Percentil 95 (p95) $< 1 500$ ms con tres nodos	p95 obtenido: 1 043,4 ms en línea base	**Superada con éxito**
Resiliencia transaccional	100 % de compensación en rechazos de guardrail	31 de 31 transacciones compensadas con código	**Cumplida estrictamente**

Meta establecida	Métrica objetivo	Resultado obtenido en pruebas	Estado de cumplimiento
		409	
Trazabilidad de eventos	Registro íntegro de eventos sin pérdida en colas	9 735 eventos publicados = 9 735 consumidos	**Cumplida estrictamente**

Limitaciones y validez

Aunque los resultados validan cuantitativamente la solidez de la arquitectura, es necesario delimitar el alcance del prototipo. La evaluación se ejecutó en una sola máquina física con cuatro núcleos lógicos; si bien los procesos y sockets operan de forma estrictamente desacoplada, comparten el bus de memoria y la CPU del host. En un despliegue distribuido geográficamente o entre instancias en la nube, las fluctuaciones de latencia inter-nodo serán mayores.

Asimismo, la gestión de la máquina de estados de la Saga en la memoria volátil del coordinador implica que una falla catastrófica de ese proceso interrumpiría las transacciones en vuelo. Para escenarios de misión crítica, la arquitectura debe evolucionar hacia un orquestador con persistencia distribuida y replicación de líderes (e.g., mediante algoritmos de consenso como Raft o bases de datos distribuidas).

Conclusiones

La implementación y evaluación experimental de NexusAI 2.0 demuestran que la arquitectura de software en sistemas distribuidos no puede validarse únicamente mediante representaciones estáticas o suposiciones teóricas. La concurrencia física, la latencia de transporte y la contención en colas son propiedades emergentes que solo se manifiestan y cuantifican cuando los componentes interactúan como procesos autónomos sometidos a cargas de trabajo reales.

El modelado formal en PlantUML proporcionó una base sólida para gobernar la complejidad del sistema: el diagrama de componentes desacopló las interfaces de servicio, el diagrama de secuencia detalló con precisión las barreras temporales entre hilos de despacho y pools de cómputo, y el diagrama de despliegue aseguró una correspondencia directa con arquitecturas modernas basadas en microservicios y contenedores.

En el plano experimental, la combinación de patrones arquitectónicos probó ser altamente efectiva:

1. **Escalabilidad verificada:** La separación de nodos sin estado y el aislamiento de hilos permitieron alcanzar un speedup de $2,89 imes$ y una eficiencia del 96,5 % al triplicar los nodos de cómputo, reduciendo la latencia p95 a un tercio del valor base.

2. **Resiliencia ante el caos:** La articulación entre el disyuntor de circuito (Circuit Breaker), mamparos (Bulkhead) y reintentos selectivos mantuvo una disponibilidad perfecta del 100 % en todos los escenarios de degradación y caída de instancias, erradicando fallos no controlados.

3. **Integridad transaccional:** La Saga orquestada demostró su capacidad para preservar la consistencia eventual, revirtiendo ordenadamente las transacciones rechazadas por reglas de negocio mediante compensaciones inversas idempotentes.

4. **Lecciones críticas de infraestructura:** Las mediciones empíricas evidenciaron que detalles de bajo nivel, como la resolución de nombres en IPv4/IPv6, pueden degradar un sistema en órdenes de magnitud si no se auditan; y que en situaciones de degradación asimétrica, la estrategia de balanceo *Least-Connections* supera abrumadoramente a *Round-Robin*, conservando más del doble del caudal transaccional.

En síntesis, NexusAI 2.0 consolida un marco arquitectónico maduro, modular y reproducible, alineado con las mejores prácticas de la ingeniería de software moderna y respaldado por evidencia experimental incontestable.

Referencias

Bass, L., Clements, P., & Kazman, R. (2022). *Software architecture in practice* (4th ed.). Addison-Wesley.

Fowler, M. (2014, 25 de marzo). *Microservices*. https://martinfowler.com/articles/microservices.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Garcia-Molina, H., & Salem, K. (1987). Sagas. *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 249–259. https://doi.org/10.1145/38713.38742

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. https://www.iso.org/standard/35733.html

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

PlantUML. (2024). *PlantUML language reference guide*. https://plantuml.com/guide

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Anexo A. Contratos HTTP del prototipo

El clúster implementa contratos REST/JSON uniformes con especificación explícita de códigos de respuesta HTTP para diferenciar estados de infraestructura frente a decisiones semánticas de negocio.

Tabla 12

Puntos de acceso HTTP expuestos por cada tipo de nodo

Nodo responsable	Método y ruta	Propósito del endpoint	Códigos de respuesta
Coordinador	`POST /task`	Inicializa y orquesta una transacción Saga completa	200 OK, 409 Conflict, 503 Service Unavailable
Coordinador	`GET /metrics`	Expone telemetría agregada de throughput y percentiles	200 OK
Coordinador	`POST /reset`	Restablece contadores internos y reinicia disyuntores	200 OK
Coordinador	`GET /dashboard.html`	Sirve la interfaz web de supervisión operativa	200 OK
Trabajador	`POST /execute`	Ejecuta una etapa cognitiva individual del pipeline	200 OK, 500 Internal Error
Trabajador	`GET /metrics`	Retorna telemetría local de hilos y percentiles del nodo	200 OK
Trabajador	`POST /chaos`	Inyecta retardos artificiales o configura fallos forzados	200 OK
Broker	`POST /publish`	Inserta un evento asíncrono en un tópico específico	200 OK, 503 Buffer Full
Broker	`GET /consume`	Extrae eventos secuenciales para persistencia y auditoría	200 OK, 204 No Content
Broker	`GET /metrics`	Reporta profundidad de colas y estadísticas de flujo	200 OK

Ejemplo de solicitud transaccional omnicanal recibida por el API Gateway:

```
POST /task HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
```

```
{
  "payload": {
    "prompt": "escalar incidente de infraestructura #42",
    "canal": "asistente_movil"
  }
}
```

Ejemplo de respuesta nominal con desglose temporal de ejecución:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "completed",
  "saga_id": "e77feb53a1",
  "steps": [
    {"step": "triage",     "worker": "w1", "attempt": 1, "latency_ms": 118.4},
    {"step": "rag",        "worker": "w2", "attempt": 1, "latency_ms": 231.7},
    {"step": "compliance", "worker": "w2", "attempt": 1, "latency ms": 134.9}
  ],
  "duration_ms": 501.8,
  "e2e latency ms": 508.2
}
```

Ejemplo de respuesta tras rechazo de guardrail y compensación inversa:

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "status": "rolled_back",
  "saga_id": "7bdadaa043",
  "failed_step": "compliance",
  "reason": "guardrail: confianza semantica < 0.90",
  "compensated_steps": 2,
  "duration_ms": 673.9
}
```

Anexo B. Extracto de la bitácora de eventos

La bitácora persistida en formato JSONL almacena cada transición del clúster con estructura de sobre estandarizada:

```
{"ts": 1794267910808.94, "topic": "events", "message": {"event": "saga.started",
"source": "coordinator", "saga_id": "ddefa5c333", "canal": "web"}}
{"ts": 1794267974934.96, "topic": "events", "message": {"event": "saga.rolled_back",
"source": "coordinator", "saga_id": "7bdadaa043", "failed_step": "compliance", "reason":
"guardrail: confianza semantica < 0.90", "compensated_steps": 2, "duration_ms": 673.93}}
{"ts": 1794267998822.08, "topic": "events", "message": {"event": "saga.completed",
"source": "coordinator", "saga_id": "9b8b797e4e", "duration_ms": 482.17}}
```

Tabla 13

Eventos registrados por tipo durante la suite completa

Tipo de evento transaccional	Volumen total auditado	Significado en el ciclo de vida del sistema
`saga.started`	1 212	Inicializaciones registradas por el API Gateway
`saga.step.completed`	3 605	Fases individuales culminadas con éxito por workers
`task.completed`	3 667	Confirmaciones de entrega procesadas por trabajadores
`saga.completed`	1 180	Transacciones cerradas con éxito integral (código 200)
`task.failed`	40	Incidentes de infraestructura inyectados en pruebas de caos
`saga.rolled_back`	31	Transacciones compensadas ordenadamente por guardrails

Anexo C. Salida de la suite de evaluación

Fragmento de la salida consolidada emitida por el ejecutor automatizado `benchmark.py` que alimenta las mediciones del informe:

```
=== escalabilidad_1_nodo (1 nodo de computo; carga: 180 transacciones, c=24) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput=  9.24 rps   wall= 19.47s   handled=100.00%
latencia e2e: avg=2469.2ms  p50=2131.3ms  p95=3185.0ms  p99=3243.4ms  max=3251.2ms

=== escalabilidad 2 nodos (2 nodos de computo; carga: 180 transacciones, c=24) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput= 18.14 rps   wall=  9.92s   handled=100.00%
latencia e2e: avg=1244.3ms  p50=1047.7ms  p95=1617.0ms  p99=1638.8ms  max=1671.3ms

=== escalabilidad 3 nodos (3 nodos de computo; carga: 180 transacciones, c=24) ===
tareas= 180  ok= 180  compensadas=   0  errores=  0
throughput= 26.74 rps   wall=  6.73s   handled=100.00%
latencia e2e: avg=844.7ms  p50=854.2ms  p95=1043.4ms  p99=1127.0ms  max=1157.9ms

=== robustez linea base (3 nodos saludables; carga: 120 transacciones, c=24) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 26.37 rps   wall=  4.55s   handled=100.00%
latencia e2e: avg=831.8ms  p50=843.5ms  p95=1033.6ms  p99=1099.5ms  max=1207.0ms

=== robustez latencia w2 (300 ms de latencia adicional inyectada en nodo w2) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput=  8.83 rps   wall= 13.59s   handled=100.00%
latencia e2e: avg=2419.7ms  p50=2546.7ms  p95=4128.4ms  p99=4945.4ms  max=5320.0ms

=== robustez_nodo_defectuoso_w2 (w2 falla sistematicamente el 100% de llamadas) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 17.87 rps   wall=  6.71s   handled=100.00%
latencia e2e: avg=1226.3ms  p50=1258.3ms  p95=1531.2ms  p99=1624.0ms  max=1649.1ms

=== robustez_guardrail (25% de transacciones rechazadas en compliance) ===
tareas= 120  ok=  89  compensadas=  31  errores=  0
throughput= 23.82 rps   wall=  5.04s   handled=100.00%
latencia e2e: avg=914.6ms  p50=913.9ms  p95=1279.2ms  p99=1377.5ms  max=1498.7ms

=== robustez_caida_w3 (w3 apagado fisicamente durante la carga activa) ===
tareas= 120  ok= 120  compensadas=   0  errores=  0
throughput= 14.55 rps   wall=  8.25s   handled=100.00%
latencia e2e: avg=1533.8ms  p50=1323.5ms  p95=2828.8ms  p99=3453.5ms  max=3648.3ms

=== robustez_recuperacion_w3 (w3 reiniciado; Circuit Breaker retorna a CLOSED) ===
tareas=  60  ok=  60  compensadas=   0  errores=  0
throughput= 23.53 rps   wall=  2.55s   handled=100.00%
latencia e2e: avg=475.2ms  p50=470.6ms  p95=518.6ms  p99=570.4ms  max=595.0ms
```

Anexo D. Reproducción del entorno

El prototipo se ejecuta íntegramente sobre la biblioteca estándar de Python 3.11+. Las herramientas adicionales (`matplotlib`, `plantuml`) se emplean de forma auxiliar para compilar gráficos y diagramas.

Tabla 14

Secuencia de comandos para reproducción integral del clúster

Paso	Instrucción en consola de comandos	Resultado esperado de ejecución
1	`python run_cluster.py --workers 3`	Inicialización de procesos y confirmación de puertos en línea
2	Navegar a `http://127.0.0.1:8088/dashboard.html`	Despliegue visual del tablero de telemetría en tiempo real
3	`python load_test.py --tasks 120 --concurrency 24`	Inyección de carga sintética con cálculo de throughput y percentiles
4	`python chaos_test.py latency --worker http://127.0.0.1:8102 --ms 300`	Inyección de retardo artificial en el trabajador w2
5	`python chaos_test.py fail --worker http://127.0.0.1:8102 --rate 1.0`	Activación forzada de fallos; apertura del Circuit Breaker en OPEN
6	`python chaos_test.py reset`	Restablecimiento de los disyuntores y normalización de nodos
7	`python benchmark.py`	Ejecución de la suite completa y exportación de métricas en `pruebas/`
8	`python ../pruebas/experimento_estrategias.py`	Evaluación comparativa de estrategias de balanceo y gráfica 05
9	`python ../diagramas/render_diagrams.py`	Regeneración automatizada de diagramas UML en PNG y SVG
10	`python build_reporte.py`	Compilación formal del documento editable `.docx`

Anexo E. Enlaces de entrega

Tabla 15

Enlaces formales de los componentes del entregable

Componente entregable	Canal y ubicación oficial
Repositorio central de código fuente	https://github.com/Alboguaci/tejiendo-redes-nexusai
Video de sustentación técnica (YouTube, no listado)	Pendiente de publicar
Informe técnico consolidado (formato PDF)	Archivo adjunto oficial en plataforma institucional